Consistency Models & the CAP Theorem - Lab Report

CSS4004 - Distributed Systems | Karbozov Baglan

Section 1: Test Results & Quorum Verification

pytest (tests/test_cap.py): **22/22 passed**. Cluster: 3 replicas, A = leaseholder; partition DC-1 = {A, B}, DC-2 = {C}.

Test case	Result
test_cp_write_succeeds_on_majority_partition	PASSED
test_cp_write_rejected_on_minority_partition	PASSED
test_cp_write_survives_one_crash_but_not_two	PASSED
test_ap_write_accepted_on_minority_partition	PASSED
test_ap_partition_recovery_lww_newer_timestamp_wins	PASSED
test_lww_decides_by_timestamp_not_by_arrival_order	PASSED
test_cp_write_reaches_isolated_node_after_heal	PASSED
test_linearizable_read_is_proxied_to_leaseholder	PASSED
test_linearizable_read_fails_when_leaseholder_unreachable	PASSED
test_follower_read_stays_available_without_leaseholder	PASSED
test_stale_read_exceeded_when_follower_lags	PASSED
test_idempotent_put_is_applied_once	PASSED
test_majority_is_n_div_2_plus_1	PASSED
test_quorum_intersection_experiment[1-1]	PASSED
test_quorum_intersection_experiment[1-2]	PASSED
test_quorum_intersection_experiment[1-3]	PASSED
test_quorum_intersection_experiment[2-1]	PASSED
test_quorum_intersection_experiment[2-2]	PASSED
test_quorum_intersection_experiment[2-3]	PASSED
test_quorum_intersection_experiment[3-1]	PASSED
test_quorum_intersection_experiment[3-2]	PASSED
test_quorum_intersection_experiment[3-3]	PASSED

Quorum verification (N = 3)

For each (R, W): the old value is on all nodes, the cluster is fully partitioned, the new value (higher timestamp) is written to W nodes, and *every* possible set of R nodes is read; the reader takes the newest timestamp. 'All reads fresh' must be true exactly when R + W > N.

R	W	N	R+W>N	All R-subsets read the newest value?	Stale read found for
1	1	3	no	no	R-nodes ['B']
1	2	3	no	no	R-nodes ['C']
1	3	3	yes	yes	-
2	1	3	no	no	R-nodes ['B', 'C']

R	W	N	R+W>N	All R-subsets read the newest value?	Stale read found for
2	2	3	yes	yes	-
2	3	3	yes	yes	-
3	1	3	yes	yes	-
3	2	3	yes	yes	-
3	3	3	yes	yes	-

Section 2: Performance Evaluation (PACELC)

1000 iterations per scenario, 3 replicas on localhost. The client is attached to follower B; Linearizable reads are proxied to the leaseholder A, Follower-Stale reads (max_staleness_ms = 5000) are answered locally. Cross-node calls carry a simulated delay of 20 ms because all nodes share one machine.

Read Mode	Target Node	Median Latency (ms)	p95 Latency (ms)	Read Availability (%)
Linearizable	Leaseholder (Remote)	21.43	21.91	100.0
Follower Stale	Local Follower	0.42	0.56	100.0

The follower read is about 51.1x faster at the median (0.42 ms vs 21.43 ms): it avoids the hop to the leaseholder (the 'L' of PACELC: when there is no partition, we trade consistency for latency).

Extra: leaseholder partitioned away from the client's follower (200 iterations each)

Read Mode	Median (ms)	p95 (ms)	Read Availability (%)
Linearizable	n/a	n/a	0.0
Follower Stale	0.42	0.55	100.0

During the partition the linearizable path refuses to answer (consistency over availability, the 'C' in PC), while bounded-stale follower reads keep being served until the staleness bound is exceeded (the 'A' in PA).

Section 3: Conceptual Questions

1. Why does R + W > N guarantee up-to-date reads, whereas R + W ≤ N allows stale reads?

A write is acknowledged by W of the N replicas and a read contacts R of them. Two subsets of sizes W and R taken from N nodes share at least R + W - N nodes (pigeonhole principle). If R + W > N that number is at least 1, so every read set contains at least one replica that holds the latest acknowledged write; because values carry a version/timestamp, the reader picks the newest and never returns an older one. This is the property that, together with a single order of writes (here: the leaseholder / majority CP writes), gives linearizable behaviour. If R + W ≤ N the two sets may be disjoint: e.g. N = 3, W = 1, R = 2 - the write lands on A, the read asks B and C, neither has it, and a stale value is returned. Our experiment in Section 1 shows exactly this: stale reads appear for every (R, W) with R + W ≤ 3 and never for R + W > 3.

2. What trade-off did the system make for writes on the minority partition: QUORUM_CP vs ALWAYS_AP?

QUORUM_CP chose consistency over availability: node C (1 of 3) can never collect the required majority of 2 ACKs, so the write is rejected (ReplicaUnavailableError, acks_received = 1) and nothing is applied. The majority side (A, B) keeps accepting writes (acks_received = 2), so there is never a split brain, but clients on the minority side get errors. **ALWAYS_AP** chose availability over consistency: C accepted the write (success = true, acks_received = 1), so both sides accepted writes and the replicas diverged. After the partition healed, hinted handoff delivered the missing writes and Last-Write-Wins (highest timestamp, ties broken by node address) made all replicas converge - at the price that the losing concurrent write is silently discarded (a Union merge would keep both). In PACELC terms: CP = PC/EC-style behaviour, AP = PA/EL-style behaviour.